A GUIDE TO WORKFLOW YOU BUILD YOURSELF

BE YOUR OWN AGENT

How to build your own dashboards, mini-apps, mockups, and visual frameworks — using AI tools you already have access to.

DAN THE DIGITAL MAN · DTDM.US

CONTENTS

Table of Contents

Introduction

Why most people are still years behind what's already possible

Chapter 1 — The Gap

Why you've been shaping your work around software instead of the reverse

Chapter 2 — Naming Your Own Workflow Problem

Finding the thing worth building

Chapter 3 — Custom Dashboards

Pulling your own data into a view built for how you think

Chapter 4 — Custom Mini-Apps

Turning a manual process into a real tool

Chapter 5 — Mocking Up Ideas Fast

From concept to clickable in an afternoon

Chapter 6 — Generating Figures and Frameworks

Visualizing your own thinking instead of hiring it out

Chapter 7 — The Enabling Layer

What's actually running underneath, in plain terms

Chapter 8 — The Iteration Loop

Why the tool you build today isn't the tool you'll use in six months

Chapter 9 — From Personal Tool to Something Others Want

Recognizing when a fix is worth packaging

Chapter 10 — Common Objections, Answered

The reasons people talk themselves out of this before trying it

Tool Categories to Know

The categories to search for, since specific products move fast

Glossary & Quick Reference

Plain-language definitions for every term used in this guide

Closing — Where to Start Monday

The one-page version of this entire guide

CHAPTER —

Introduction

Why most people are still years behind what's already possible.

There's a specific kind of gap opening up right now, and most people don't know it exists yet. It isn't a gap in access — the tools are available to almost anyone with a laptop and a subscription. It's a gap in imagination. People still think of "customizing their workflow" as picking the right template, the right app, the right subscription tier. They shop for software that gets them 80% of the way to what they actually need, then quietly adapt themselves to cover the other 20%.

That trade-off used to be necessary. Building your own tool meant hiring a developer, learning to code, or living with a rough spreadsheet forever. That's no longer true. Today, describing a problem clearly to an AI model is most of the work. The rest — a working dashboard, a small app, a visual mockup, a diagram of your own thinking — can be produced in the time it used to take to evaluate three competing SaaS products.

This guide is not a technical manual. It doesn't assume you can code, and it isn't trying to turn you into a developer. It's about a shift in posture: from "which existing tool is closest to what I need" to "what does the tool I actually need look like, and how fast can I have it." Every example in this guide is a real tool built exactly that way — not a hypothetical, not a demo.

What this guide covers

Four categories of things you can build for yourself right now, each with a real working example: dashboards that show your own data the way you actually think about it, small apps that replace a manual process, visual mockups you can go from idea to clickable prototype with in an afternoon, and figures or frameworks that turn your own reasoning into something you can look at instead of just hold in your head.

There's also a short, deliberately light chapter on what's running underneath these tools — the models, the hosting, the connections between services. You don't need to become an expert in it. You need to know it exists, know roughly what each piece does, and know that none of it is gatekept the way it used to be.

How to use this guide

Read it in order once, since chapter 2 sets up an exercise the later chapters build on. After that first pass, treat it as a reference — when a specific kind of friction shows up in your own work, come back to whichever chapter matches its shape: a visibility problem is chapter 3, a repeated manual task is chapter 4, an untested idea is chapter 5, and reasoning you've never made explicit is chapter 6.

Each of those four chapters ends with a short worksheet — a few blanks to fill in, not a form to overthink. The worksheet is the actual deliverable of each chapter. Everything above it is context for filling those blanks in accurately.

None of this requires reading the whole guide before building anything. If chapter 3 already matches a problem you have, skip ahead, fill in that worksheet, and build the thing. Come back for the rest when the next kind of friction shows up.

1

The Gap

Why you've been shaping your work around software instead of the reverse.

CHAPTER 1

The Gap

Why you've been shaping your work around software instead of the reverse.

Every piece of software you've ever used was built by someone guessing at what a large number of different people might need. A project management tool has to work for a marketing team, a construction company, and a solo freelancer all at once. To do that, it has to generalize. Generalizing means compromise. The tool you end up with is never quite shaped like your actual workflow — it's shaped like the average of everyone's workflow, with your specific case bent to fit.

Most people accept that compromise as the cost of doing business. You learn the tool's vocabulary instead of using your own. You build workarounds inside someone else's data model. You pay a monthly fee for 90% of a feature set you'll never touch, because the 10% you do need only comes bundled that way.

What's changed is the cost of the alternative. It used to be that building something custom meant a development budget measured in thousands of dollars and weeks of back-and-forth with a contractor. Now the same output — a working dashboard, a small app, a visual prototype — can come from a clear description and an AI model doing the assembly. The skill that matters has shifted from "can you write code" to "can you describe exactly what you need."

What people still assume, incorrectly

- That building something custom requires learning to program first.
- That it will take significant time — days or weeks, not hours.
- That it will cost real money in tools, hosting, or developer fees.
- That the result will be fragile, ugly, or embarrassing to actually use.
- That this only applies to "tech people," not to their own field or workflow.

None of these are true anymore, and the rest of this guide exists to prove it with working examples rather than arguments. Every chapter from here on shows a specific category of thing you can build, followed by a real tool built exactly that way.

The old model versus the current one

Old assumption	What's actually true now
Custom software needs a developer.	A clear description plus an AI model produces working software directly.
Building something takes weeks.	A working first version can come together in a single sitting.
Hosting and infrastructure cost real money.	Free or near-free hosting covers nearly any personal tool.
You need to learn to code to be taken seriously.	Describing the problem precisely matters more than writing the syntax yourself.
Only "tech people" build their own tools.	The barrier that used to separate technical and non-technical people has largely collapsed for this category of work.

2

Naming Your Own Workflow Problem

Finding the thing worth building — before you build anything.

CHAPTER 2

Naming Your Own Workflow Problem

Finding the thing worth building — before you build anything.

The biggest obstacle isn't technical. It's that most people don't have a clear name for the thing that's actually annoying them. They feel friction — a task that takes longer than it should, information scattered across three places, a decision they keep making from memory instead of from a clear view — but they've never stopped to define it as a problem with a shape.

A well-named problem is most of the way to a built tool. "I wish things were more organized" isn't buildable. "I forget which of my subscriptions I'm actually using, and I want a single view of what I'm paying for and what I've downloaded" is buildable — and it's a real example later in this guide.

Where to look for the problem

- Anything you currently track in your head, or re-derive from memory each time.
- Anything you check in more than one place to get the full picture.
- Any spreadsheet you've built and abandoned because keeping it updated became its own job.
- Any decision you make the same way, repeatedly, without a tool that helps you make it.
- Any idea you've had that you couldn't picture clearly enough to explain to someone else.

TRY THIS — Name one

Pick one task or piece of information you dealt with in the last week that took more effort than it should have. Write one sentence describing exactly what a tool would need to show you or do for you to remove that friction. That sentence is the starting brief for chapters 3 through 6.

Notice what that exercise does not ask for: no technical spec, no list of features, no mention of what platform it should run on. Just a clear description of the problem, in your own words. Everything after this point is about turning that sentence into something real.

3

Custom Dashboards

Pulling your own data into a view built for how you think, not a vendor's defaults.

CHAPTER 3

Custom Dashboards

Pulling your own data into a view built for how you think, not a vendor's defaults.

A dashboard, at its core, is just a view: data pulled from somewhere, arranged so you can see it at a glance. Every SaaS product ships one, and every one of them is built around what the vendor assumed you'd want to look at. The moment your actual priority doesn't match their default layout, you're stuck — either living with the mismatch or paying for a higher tier that lets you "customize," inside limits they still set.

A custom dashboard skips that negotiation entirely. You decide what data matters, you decide how it's grouped, and you decide what counts as worth flagging. The technical piece — pulling data from a source and rendering it as a page you can look at — is the easy part now. The hard part was always deciding what you actually wanted to see, and that part was never technical to begin with.

The most common source of dashboard-worthy data most people already have sitting untouched: their own email. Purchase confirmations, membership renewals, receipts, delivery notices — all of it structured enough to extract, none of it surfaced anywhere useful by default.

REAL EXAMPLE — Purchase & Subscription Tracker

A dashboard built with Google Apps Script that scrapes Gmail for purchase confirmations, specifically flagging memberships and subscriptions. Rather than forgetting what's been paid for and letting a program lapse unused, the dashboard surfaces every active purchase in one place, and automatically pulls any downloadable files tied to those purchases. The problem it solves: money already spent on programs, tools, or content that quietly gets forgotten because nothing was ever tracking it in one place.

Why this is a dashboard problem, not an app problem

The distinction matters for chapter 4. A dashboard's job is to show you something clearly — it's read-first. You're not entering new data into it or triggering actions from it, you're using it to see a pattern or a status you couldn't easily see before. If your workflow problem is primarily "I can't see this clearly," you're building a dashboard. If it's "I have to manually do this same multi-step thing over and over," that's chapter 4.

Getting started on your own

1. Identify one source of data you already have but never see assembled — email, a spreadsheet you already keep, a set of accounts you check individually.
2. Describe, in plain language, what you want pulled out and how you want it grouped or flagged.
3. Use a script (Google Apps Script if the source is Gmail or Sheets) to do the pulling, and let an AI model write that script from your plain-language description.
4. Render the result as a simple page or spreadsheet view — it doesn't need to be visually elaborate to be useful.

Variations on the same idea

The purchase tracker is one instance of a much larger pattern: any inbox, spreadsheet, or account you already have is a potential dashboard source. The same approach applies to a calendar you want summarized differently than the default view, a set of bills or renewals you want flagged before they're due, or a batch of files you keep meaning to organize but never do. The specific source changes; the shape of the solution — pull, structure, surface — stays the same.

TRY THIS — Dashboard worksheet

Fill in three blanks: The data source I already have but never see assembled is ____. What I want pulled out and flagged is ____. The view I actually want to look at — a list, a grid, grouped by category or by date — is ____. That's the full brief. Hand it to an AI model as-is.

4

Custom Mini-Apps

Turning a spreadsheet-and-willpower process into a real tool with inputs, logic, and output.

CHAPTER 4

Custom Mini-Apps

Turning a spreadsheet-and-willpower process into a real tool with inputs, logic, and output.

A mini-app is one step past a dashboard: it doesn't just show you something, it does something. It takes an input, applies logic, and gives you an output — a lookup, a calculation, a transformation. This is the category where "I keep doing this manually" turns into "I built a small tool that does this for me now."

The instinct to underestimate this category is strong, because a real app sounds like it requires real infrastructure: a domain, a host, a database, a repo to manage it all in. It does require those things — but every one of them is now either free, nearly free, or already something you have an account for. The assembly is the part AI tools now handle; the pieces themselves were never the expensive part.

REAL EXAMPLE — Franchise Timeline Lookup

A website on a custom subdomain, hosted on Railway, with the repo stored on GitHub and Cloudflare sitting in front of it. The app lets you look up any character or series name and returns a full timeline for that franchise, drawing its data from IMDB in the background. Four services working together — domain, host, repo, CDN — assembled into one tool that does something no single existing product does for that specific use case.

What's actually happening under the hood

- A custom subdomain points at a small hosted application — this is naming, not building.
- Railway runs the actual application code, so nothing has to run on a personal machine.
- GitHub stores the code as a repository, giving a version history and a place to reference or update it from.
- Cloudflare sits in front for speed, reliability, and routing.
- IMDB is queried as a data source in the background — the app's job is fetching and presenting, not generating that data itself.

None of these pieces is exotic. Each one is a service you either already have or can create an account for in minutes, and increasingly, an AI model can write the connecting code between them from a plain description of what you want the app to do.

Getting started on your own

1. Write down the input, the logic, and the output in one sentence: "I give it X, it does Y, it returns Z."

2. Decide if it needs to be reachable from anywhere (a real hosted app) or just needs to run when you open it (a local file) — this determines whether you need hosting at all.

3. If it needs hosting, a platform like Railway plus a GitHub repo is enough for most personal tools — no need to over-provision infrastructure for a tool built for one user.

4. Describe the logic to an AI model in plain terms and let it produce the working code; test it against real inputs before trusting the output.

Variations on the same idea

A lookup tool pulling from a public data source is one shape this takes. Others are just as common: a calculator that applies your own specific formula instead of a generic online version, a converter between two formats you deal with regularly, or a small tool that takes a messy input and returns it cleaned up and structured the way you actually need it. The common thread is always input, logic, output — only the logic changes.

TRY THIS — Mini-app worksheet

Fill in the sentence: I give it ____, it does ____ to it, and it gives back ____. If any blank is hard to fill in specifically, the problem isn't defined clearly enough yet — go back to chapter 2 before building anything.

5

Mocking Up Ideas Fast

Going from concept to something clickable, before you ever write a formal spec.

CHAPTER 5

Mocking Up Ideas Fast

Going from concept to something clickable, before you ever write a formal spec.

Most ideas die in the gap between having them and being able to show them to anyone — including yourself. An idea that lives only as a description in your head is easy to talk yourself out of, easy to misjudge, and impossible to react to the way you'd react to something you can actually see and use. A mockup closes that gap.

This category is different from a mini-app in one important way: it doesn't need to be built for the long term, hosted anywhere, or connected to real data. Its entire job is to let you interact with an idea for an afternoon, decide whether it's worth developing further, and throw it away without cost if it isn't.

REAL EXAMPLE — Visual Dice-Rolling App

A custom, visual dice-rolling app built with Claude Design, generating a single HTML file that opens and runs locally — no server, no hosting, no account beyond the Claude subscription already in use. The entire tool went from idea to something clickable and working in a single sitting, with zero additional infrastructure or cost.

That's the pattern worth internalizing: a working, visual, interactive prototype requiring nothing beyond a tool you're already paying for, produced fast enough that trying an idea costs you an afternoon instead of a commitment.

What makes something mockup-ready

- You can describe what it looks like and how someone would interact with it, even roughly.
- You want to react to it — use it, click through it, show it to someone — before deciding if it's worth building for real.
- It doesn't yet need real data, real accounts, or to be reachable by anyone but you.
- Speed matters more than polish at this stage; polish is what you add after you know the idea survives contact with reality.

Getting started on your own

1. Describe the idea as if you were explaining it to someone who will build it — what it shows, what you click, what happens next.
2. Ask for a single self-contained file you can open locally — this removes hosting entirely from the equation while you're still deciding if the idea is worth pursuing.
3. Use it. Click through it the way an actual user would, not the way you imagined it in your head — this is where most flawed ideas reveal themselves.
4. Decide from there: refine the mockup, promote it to a real mini-app per chapter 4, or let it go having cost you almost nothing.

Variations on the same idea

The dice app is a small, self-contained example, but the same approach scales to anything you want to react to before committing: a rough layout for a product idea, an interactive version of a game mechanic, a simple simulator for a decision with several variables. The size of the idea doesn't change the approach — describe it, get a working version fast, react to the real thing instead of the idea of it.

TRY THIS — Mockup worksheet

Describe your idea in three parts: what someone sees first, what they do with it, what happens as a result. Ask for a single file that runs locally with no setup. Open it and use it exactly as a real user would, not the way you pictured it in your head.

6

Generating Figures and Frameworks

Turning your own thinking into something you can look at, instead of just hold in your head.

CHAPTER 6

Generating Figures and Frameworks

Turning your own thinking into something you can look at, instead of just hold in your head.

Some of the most useful things you can build aren't apps at all — they're visual representations of how you think about a problem. A framework that lives only in your head is useful to you and unusable by anyone else, including a future version of yourself who's forgotten the details. Turning it into a figure — a matrix, a diagram, a flow — is what makes it reusable, explainable, and easy to revise.

This used to mean hiring a designer to translate your thinking into something visual, or living with a rough whiteboard photo forever. AI tools now handle the translation directly: you describe the structure of your thinking — the axes, the categories, the relationships — and get back a clean visual you can iterate on immediately, no design skill required on your end.

Where this shows up

- Turning a decision you keep making intuitively into an explicit matrix, so the logic is visible and repeatable.
- Mapping a process with multiple stages or branches into a flow diagram instead of a paragraph someone has to re-read to follow.
- Building a comparison framework — options against criteria — instead of holding the tradeoffs in your head each time the choice comes up.
- Producing a visual you can hand to someone else and have them understand your reasoning in seconds rather than a conversation.

The value isn't the picture itself — it's what making the picture forces you to do. You can't turn fuzzy thinking into a clean diagram. The act of trying reveals which parts of your framework are actually solid and which parts you'd been gesturing at without ever fully defining.

Getting started on your own

1. Pick a piece of reasoning you use repeatedly — a way you evaluate options, a process you explain often, a set of categories you sort things into.

2. Describe its structure out loud or in writing: what are the axes, stages, or categories, and how do they relate.

3. Ask an AI design tool to render that structure visually — a matrix, a diagram, a flowchart — and treat the first version as a draft, not a final answer.

4. Revise based on what looks wrong once it's visual — this step usually catches gaps the purely-mental version hid.

TRY THIS — Framework worksheet

Name a piece of reasoning you use more than once. List its parts — axes, stages, or categories — as plainly as possible. Ask for a visual version of exactly that list. Look at it, and notice which part feels wrong or incomplete now that it's in front of you instead of in your head.

7

The Enabling Layer

What's actually running underneath these tools, in plain terms — just enough to know it exists.

CHAPTER 7

The Enabling Layer

What's actually running underneath these tools, in plain terms — just enough to know it exists.

This chapter is deliberately short. The point of this guide is what you can build, not how the underlying pieces work in detail — but it helps to know the names of the layer that makes everything in chapters 3 through 6 possible, so none of it feels like a black box.

AI models

The part that does the actual writing, coding, and designing when you describe what you want. You don't need to know how a model works to use one effectively — you need to describe your problem clearly, the same way you'd brief a capable person you'd just met.

Hosting

If a tool needs to be reachable by more than just you on your own machine, it needs somewhere to run. Modern hosting platforms exist specifically to make this close to automatic — you connect a code repository, and the platform handles running it, no server administration required on your end.

Repositories

A place your code lives with a history of changes attached to it. Its practical benefit for a personal tool is simple: if something breaks after a change, you can see exactly what changed and roll it back.

Connections between services (OAuth)

When a tool needs to read your email, your calendar, or another account you already have, it connects through a permission system — you're granting a specific, revocable level of access, not handing over your password. This is the same mechanism behind "Sign in with Google" buttons you've already used dozens of times, just pointed at something you built instead of something a company built.

That's the entire layer. None of it requires you to become fluent in it — only to recognize the names when an AI tool mentions them, and to know that each one is a small, well-understood piece, not a specialized skill gatekept behind a computer science degree.

8

The Iteration Loop

Why the tool you build today isn't the tool you'll be using in six months — and why that's fine.

CHAPTER 8

The Iteration Loop

Why the tool you build today isn't the tool you'll be using in six months — and why that's fine.

Software you buy gets updated on someone else's schedule, in directions you don't control, at a pace that's either too slow for what you need or moving features you rely on. Software you build yourself has the opposite problem in the best possible way: it changes exactly when you decide it should, and in exactly the direction your actual workflow has moved.

This is the part people underestimate most. The value of a self-built tool isn't just the first version — it's that the second, third, and tenth versions cost almost nothing to produce, because the same process that built version one builds every version after it. A dashboard that showed the right thing in January can show a different right thing in June, the moment your priorities shift, without waiting on a vendor's roadmap.

What the loop actually looks like

1. Use the tool in real conditions, not just the moment you built it.
2. Notice friction — a field you keep wishing existed, a view that no longer matches how you actually work, a step that's become unnecessary.
3. Describe the change the same way you described the original build: plainly, as a problem to solve.
4. Rebuild that piece. Because you own it end to end, there's no vendor ticket to file and no feature request to wait on.

Treat every tool from chapters 3 through 6 as a first draft, not a finished product. The habit worth building isn't "build the perfect dashboard once" — it's "notice friction and close it quickly, as a routine, the same way you'd adjust a physical workspace that wasn't quite right."

9

From Personal Tool to Something Others Want

Recognizing when a fix built for you is worth packaging for someone else.

CHAPTER 9

From Personal Tool to Something Others Want

Recognizing when a fix built for you is worth packaging for someone else.

Not every tool you build for yourself is meant to stay personal. Some of them solve a problem that's genuinely common — you just happened to be the one who got tired of it first. Recognizing that moment is a separate skill from building the tool in the first place, and it's worth being deliberate about instead of accidental.

Signs a personal tool might be worth packaging

- You've mentioned it to more than one person and gotten some version of "wait, I need that."
- The problem it solves isn't unique to your specific job, hobby, or situation — it's a version of a problem lots of people have.
- You keep improving it anyway, for your own use, which means most of the packaging work is already half-done.
- You can describe who else has this exact problem in one sentence, without hedging.

None of this requires a big leap. The distance between "tool I use" and "tool someone else could use" is usually smaller than it looks — often just making the setup less specific to your own accounts and adding the minimum explanation someone else would need to get started. It's worth treating that packaging step as its own small project, not something you back into by accident.

This is also the point where the proof matters more than any pitch. A tool that's actually been solving your own real problem, for real, is a stronger case for anyone else's interest than any amount of description of what it could theoretically do.

CHAPTER 10

Common Objections, Answered

The reasons people talk themselves out of this before trying it.

None of the objections below are unreasonable — they were true for a long time. What's changed is that they stopped being true recently enough that most people haven't updated their assumptions yet. This chapter exists to close that gap directly.

"I'm not technical."

Nothing in this guide requires writing code yourself. The skill that matters is describing a problem clearly — the same skill you'd use briefing a new hire. The AI model handles the translation from description to working tool.

"This will take too long."

Every real example in this guide was built in a single sitting, not over days or weeks. The time cost that used to exist — learning a language, setting up an environment, debugging alone — has largely moved onto the AI model doing the building, not onto you.

"It'll be expensive."

Every example in this guide runs on infrastructure that's free or already paid for as part of an existing subscription. A personal tool for one user doesn't need enterprise-grade hosting or a development budget.

"It'll be fragile or break constantly."

A personal tool only has to keep working for you, under conditions you control. It doesn't need to survive thousands of simultaneous users or edge cases you'll never hit. That's a dramatically lower bar than commercial software has to clear.

"This doesn't apply to my field."

The pattern in this guide — dashboard, mini-app, mockup, framework — is generic on purpose. It applies equally to a workflow in construction, healthcare, retail, or a creative field. The examples happen to be built by one person, but the shape of the problem they solve isn't specific to that person's industry.

CHAPTER —

Tool Categories to Know

You don't need a specific product recommendation — you need to know which category to reach for.

This guide deliberately avoids anchoring itself to one specific product, because the exact tools available change faster than a printed guide can track. What doesn't change as quickly is the category of tool each job calls for. Knowing the category is enough to search for a current option when you're ready to build.

Category	What it's for
AI chat / design assistants	Turning a plain-language description into working code, a visual mockup, or a structured document — the starting point for nearly everything in this guide.
Cloud hosting platforms	Running a tool so it's reachable from anywhere, without managing a server yourself. Needed only once a mini-app or dashboard has to be reachable by more than just you, locally.
Code repositories	Storing your tool's code with a change history attached, so updates can be tracked and reversed.
Automation / script builders	Connecting existing accounts — email, spreadsheets, calendars — and pulling or moving data between them automatically.
Domain and DNS management	Giving a hosted tool its own address, and pointing that address at the right place.

When you're ready to build, search for "current [category] 2026" rather than relying on a name printed here — the specific leading product in each category is one of the faster-moving details in this entire guide.

CHAPTER —

Glossary & Quick Reference

Plain-language definitions for every term used in this guide.

AI model	The system that turns a plain-language description into working code, a visual design, or a written result. This guide treats it as a capable collaborator, not a search engine.
Dashboard	A read-first view that pulls existing data into one place, arranged the way you actually think about it.
Mini-app	A small tool that takes an input, applies logic, and returns an output — something that does a task rather than just displaying information.
Mockup / prototype	A fast, disposable version of an idea, built to be reacted to rather than relied on long-term.
Framework / figure	A visual representation of reasoning you already do — a matrix, diagram, or flow — made explicit and reusable.
Hosting	A service that runs your tool so it's reachable without your own computer needing to stay on. Used only when a tool needs to be reachable by more than just you, locally.
Repository (repo)	A stored, versioned copy of your tool's code, with a history of every change — lets you see what changed and undo it if needed.
OAuth / connection	The permission system that lets a tool you built access another account — email, calendar, and so on — without handing over a password. The same mechanism behind any "Sign in with Google" button.
Script	A short, purpose-built program that performs one task — pulling data, transforming a file, automating a repeated step — usually the engine behind a dashboard or mini-app.
Custom domain / subdomain	A web address you control, pointing at a tool you built rather than a page hosted by someone else's platform.

CHAPTER —

Where to Start Monday

The one-page version of this entire guide.

If none of the previous chapters turn into action, this page is the fallback. One workflow problem, named clearly, is worth more than a full understanding of everything covered here.

The four questions

1. What's one thing you currently track in your head, or check in more than one place, that a single clear view would fix? — that's your dashboard.
2. What's one manual, repeated process you do the same way every time? — that's your mini-app.
3. What's one idea you've been carrying around without ever seeing it as something real? — that's your mockup.
4. What's one piece of your own reasoning that only exists in your head? — that's your framework.

Pick one. Write the one-sentence brief from chapter 2. Build it this week, not eventually. The gap described in chapter 1 doesn't close by reading about it — it closes the first time you build something that didn't exist before you needed it.

DAN THE DIGITAL MAN

dtdm.us